CEUB - FATECS
Programação Orientada a Objetos - POO
Object Oriented Programming - OOP
Python - Prof. Barbosa

1- Conceitos

A Programação Orientada a Objetos (POO) é um paradigma de programação que se baseia nos conceitos de classes, objetos, herança, polimorfismo, entre outros.

Com a POO, é possível modelar o mundo real em objetos e estruturas de dados.

Esse paradigma é amplamente utilizado em linguagens de programação modernas, como Python, Java, C#, C++entre outras.

As **classes** possuem:
- propriedades ou características (atributos) e
- compartamentos (métodos), que são funções que realizam ações relacionadas a esses objetos.

Alan Kay inventou a Linguagem de programação **SmallTalk** e é um dos pais do conceito de programação orientada a objetos.

A programação orientada a objetos tem suas origens na **década de 1960**, mas somente em meados da **década de 1980** se tornou o principal paradigma de programação utilizado na criação de software.

- O **paradigma POO** foi desenvolvido como forma de tratar:
O rápido **aumento no tamanho e complexidade** dos sistemas de software; e
Facilitar a modificação (manutenibilidade) desses sistemas grandes e complexos no tempo.

- Programação procedural versus programação orientada a objetos

No **paradigma de programação procedural:**
O foco é na escrita de **funções** ou procedimentos que operam sobre os dados.

Na **programação orientada a objetos:**
O foco é na criação de **objetos** que contém tanto os dados quanto as funcionalidades.

A **definição** de**:**
Objeto corresponde a algum **objeto** ou **conceito** no mundo real e
As funções que operam sobre tal objeto correspondem as formas que os objetos reais interagem.

2- Classes e Objetos
2.1- Classes
São a base da POO e representam a estrutura que define um objeto.
Uma classe é um modelo que contém:
- atributos (dados) e
- métodos (funções)
que podem ser usados pelos objetos dessa classe.

Exemplo:
A classe Carro:
Os atributos (propriedades): modelo, ano, cor etc e
Os métodos (ações): acelerar, frear, ligar, parar etc.

2.2- Objetos
São as instâncias das classes e representam os elementos reais que serão utilizados pelo programa.

Exemplo 1:
Um objeto "carro_1" da classe "Carro":
Os atributos (propriedades) modelo = "HB 20", ano = 2024, cor = "vermelho".
Exemplo 2:
Um objeto "carro_2" da classe "Carro":
Os atributos (propriedades) modelo = "Corolla", ano = 2025, cor = "prata".

3. Declaração de Classe

A declaração de uma classe é feita utilizando a palavra reservada "class", seguida do nome da classe e dois pontos.

- Sintaxe de declaração de classe

```
class NomeClasse:                       # Classe
    def __init__(self, valor1, valor2):  # Construtor
        # atributos da classe
        self.atributo1 = valor1
        self.atributo2 = valor2

    def nome_metodo(self, parametros):  # Métodos
        # corpo do método
        [return um_valor]               # Opcional
```

Onde:
- class é a palavra-chave para definir uma classe
- NomeClasse é o nome da classe, em CamelCase (iniciando com letra maiúscula)
- __init__ é o método construtor, que é executado quando um objeto da classe é criado
- self é uma referência ao objeto que está sendo criado
- valor1, valor2 são os parametros do construtor
- self.atributo1 = valor1, define um atributo da classe com o nome atributo1 que recebe valor1
- def nome_metodo(self, parametros_metodo), define um método da classe
- parametros são os parâmetros que o método recebe
- return um_valor, retorna um valor do método, opcional

- Sintaxe do objeto

```
nome_objeto = NomeClasse(argumetos_do_construtor)
```

Onde:
- nome_objeto é o nome do objeto que está sendo instanciado (criado)
- NomeClasse é o nome da classe que está sendo usada, chama o método __init__
- argumetos _construtor são os argumetos (dados) que são passados para o construtor da classe

4- Classe Canino

Uma classe representa uma categoria de objeto, por exemplo, a classe carro envolve todos os tipos de carro, mas cada carro ou objeto implementa seus atributos e métodos.

Objetos são abstrações computacionais que representam entidades, com as qualidades (atributos) e as ações (métodos) que elas podem realizar.

A classe é a estrutura básica do paradigma de orientação aos objetos, que se refere ao seu tipo, um modelo a partir do qual esses objetos serão criados.

Na Figura 2, você pode observar uma estrutura de classes.

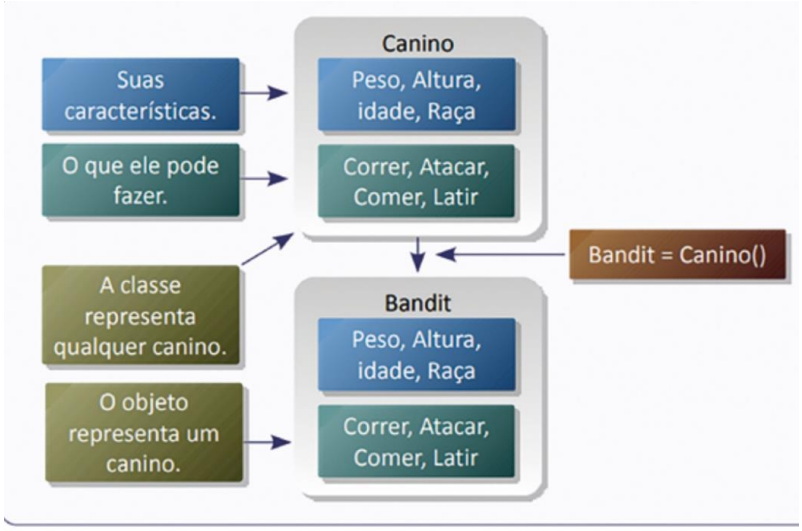


Figura 2. Exemplo de uma estrutura de classes.
Fonte: Borges (2014, p. 109).

Na Figura 2:
- A classe Canino descreve as características e ações dos caninos em geral,
- O objeto Bandit representa um canino específico.
- Os atributos envolvem as estruturas de dados que armazenam informações sobre o objeto;
- Os métodos, por sua vez, são funções associadas ao objeto, que descrevem como ele se comporta.

5- As definições de classes

Podem aparecer em qualquer lugar em um programa, mas são geralmente perto do **início** (após os comandos import).

As regras de **sintaxe** para a definição de uma classe são as mesmas de outros comandos compostas:
- Um cabeçalho que começa com a palavra-chave **class**,
- Seguido pelo **nome da classe**, e
- Terminando com dois **pontos**.

Se a primeira linha após o cabeçalho de classe é um string, ele se torna o docstring da classe, e será reconhecido por diversas ferramentas (**docstrings** funciona com funções também).

- Método especial ou mágico ou inicializador ou dunders:
Toda classe deve ter um **método** construtor com o nome __init__.

Esse método de **inicialização**, muitas vezes referido como o **construtor**, é chamado automaticamente sempre que uma nova instância de Point é criada.

Ela dá ao programador a oportunidade de configurar os atributos necessários dentro da nova instância, dando-lhes seus estados (valores **iniciais)**.

O parâmetro **self** (pode escolher qualquer outro nome) é definido automaticamente para **referenciar** o objeto recém-criado que precisa ser inicializado.

- Classes no Python

As classes são o principal recurso da POO, que tem como objetivo:
- criar modelos que representem objetos da vida real,
- com atributos e métodos que atendam a essa representação (LUTZ; ASCHER, 2007).

- Sintaxe:

```
class NomeClasse:
    def __init__ (self, p_1, p_2):            # Método construtor
        self.nome_atributo1 = p_1              # Atributos
        self.nome_atributo2 = p_2
    def get_nome_atributo1(self):              # Modelo do método get (retorna valor do atributo)
        return self.nome_atributo1
    def set_nome_atributo1(self, novo_valor):  # Modelo do método set, altera valor (atributo)
        self.nome_atributo1 = novo_valor
    def outro_metodo(self):                    # Outros métodos da classe (métodos fundionais)
        # script
        return ...

if __name__ == '__main__':                     # mai <tab> (tecla de atalho)
    nome_objeto1 = NomeClasse(a_1, a_2)  # Cria (instancia) o primeiro objeto da classe
    nome_objeto2 = NomeClasse(a_1, a_2)  # Cria (instancia) o segundo objeto da classe
    . . .
    r = nome_objeto1.get_nome_atributo1()  # Consulta um atributo, solução 1
    print("Atributo:",r)
    print("Atributo:", nome_objeto1.get_nome_atributo1()) # Consulta um atributo, solução 2
    nome_objeto1.set_nome_atributo1(novo_valor)           # Altera o valor do atributo
```

Exemplo:

Utilize a linguagem Python e crie uma classe para representar animais de estimação, com os seguintes atributos (características):
- nome,
- espécie,
- idade

```
class AnimalEstimacao:
    def __init__ (self, nome, espécie, idade):    # Método construtor
        self.nome = nome                          # Atributos
        self.especie = especia
        self. idade = idade
    def get_nome(self):                  # Modelo do método get (retorna valor do atributo)
        return self.nome
    def set_nome(self, novo_nome):   # Modelo do método set, altera valor (atributo)
        self.nome = novo_nome
    def outro_metodo(self):              # Outros métodos da classe (métodos fundionais)
        # script
        return ...

if __name__ == '__main__':                     # mai <tab> (tecla de atalho)
    # Cria (instancia) o primeiro objeto da classe
```

```
# nome_objeto = NomeClasse( Dados )
animal1 = AnimalEstimacao(“Toddy”, “cachorro”, 5)
# Cria (instancia) o segundo objeto da classe
animal2 = AnimalEstimacao (“Bob”, “gato”, 7)
. . .
r = animal1.get_nome()  # Consulta um atributo, solução 1
print("Atributo:",r)
# print("Atributo:", nome_objeto.nome_metodo()) # Chama um método com return
print("Atributo:", animal1.get_nome())          # Consulta o valor do atributo
# nome_objeto.nome_metodo(novo_valor)          # Chama um método sem return
animal1.set_nome(“Rex”)                        # Altera o valor do atributo
```

O self que aparece na frente de cada atributo e no primeiro parâmetro do método __ init __() se refere ao objeto criado, assim, self.especie define um atributo da classe que é a espécie do objeto animal de estimação.

Note que especie e self.especie são distintos:
especie é uma variável; e
self.especie é um atributo da classe.

Até aqui apenas se definiu atributos para a classe.
Agora serão desenvolvidas algumas ações e, para isso, se utiliza a definição de métodos para a classe AnimalEstimacao, como correr, brincar e comer.

As classes implementam atributos e métodos, sendo que estes executam ou implementam uma funcionalidade, assim como as funções.
Portanto, a classe é um elemento que carrega diversas funções dentro de si.

- A POO é baseada em quatro pilares principais:

1. **Abstração**: é a capacidade de representar um objeto do mundo real em código, com todas as suas características relevantes, e ignorando as que não são importantes.

2. **Encapsulamento**: é a capacidade de esconder os detalhes internos de um objeto, expondo apenas a interface pública para que os outros objetos possam interagir com ele.

3. **Herança**: é a capacidade de criar novas classes a partir de classes já existentes, herdando suas propriedades e métodos e adicionando ou modificando o comportamento.

4. **Polimorfismo**: é a capacidade de um objeto ser referenciado de diferentes maneiras, e responder de forma diferente dependendo do contexto em que é utilizado.

Esses conceitos de POO são aplicados em várias linguagens de programação, como Python, Java, C++ e outras.
O código é organizado em classes, que são modelos para a criação de objetos.
Cada objeto criado a partir de uma classe é uma instância dessa classe, e possui as mesmas propriedades e métodos definidos na classe.

A POO tem várias vantagens, como a reutilização de código, a modularidade, a facilidade de manutenção e a escalabilidade. Além disso, é uma abordagem muito utilizada em programação de jogos, aplicativos e sistemas complexos.

- Glossário

atributo
Um dos itens nomeados de dados que compõem uma instância.

classe
Um tipo de composto definido pelo usuário. Uma classe também pode ser pensada como um **modelo** para os objetos que são instâncias da mesma. (O iPhone é uma classe. Até dezembro de 2010, as estimativas são de que 50 milhões de instâncias tinham sido vendidas!)

construtor
Cada classe tem uma "fábrica", **chamada** pelo mesmo nome da classe, por fazer novas instâncias. Se a classe tem um *método de **inicialização***, este método é usado para obter os atributos (ou seja, o estado) do novo objeto adequadamente configurado.

instância
Um objeto cujo tipo é de alguma classe. Instância e objeto são usados como **sinônimos**.

instanciar
Significa criar uma instância de uma classe e executar o seu método de inicialização.

linguagem orientada a objetos
Uma linguagem que fornece recursos, como as **classes** definidas pelo usuário e **herança**, que facilitam a programação orientada a objetos.

Método
Uma **função** que é definida dentro de uma definição de classe e é chamado em instâncias dessa classe.

método de inicialização
Um método especial em Python (chamado **__init__**) que é chamado automaticamente para configurar um objeto recém-criado no seu estado inicial (padrão de fábrica).

objeto
Um tipo de **dados** composto que é frequentemente usado para modelar uma coisa ou conceito do mundo real.
Ele agrupa os **dados** e as **operações** que são relevantes para esse tipo de dados. Instância e objeto são usados como sinônimos.

Programação orientada a objetos
Um estilo poderoso de programação em que os dados e as operações que os manipulam são organizados em classes e métodos.

Prof. Barbosa.